Entity Framework Core: Die 10 häufigsten Stolperfallen

Konferenzvortrag (45-60 Minuten)

Speaker: Olaf Lischke

kontaktdaten.vcf

macht .NET, seit es .NET gibt

versucht, Projekte und Seminare zu ko

singt Tenor in Chören und Musikprojekten

zockte schon auf dem ZX 81, heute ausschließlich auf PC

fotografiert, seit er eine Kamera halten kann

fliegt, wenn Wetter und Zeit es zulassen (TMG/SEP)

Abfragen

Stolperfalle #1: N+1-Queries

Problem:

- LINQ-Query mit Lazy Loading erzeugt hunderte SQL-Statements
- Performance-Einbruch bei größeren Datenmengen
- Häufig unbemerkt in Produktionsumgebungen

Best Practice:

- Eager Loading mit `.Include()` verwenden
- Selektiv und bewusst einsetzen
- Bei Bedarf DTOs/Projektionen mit `.Select()` nutzen

Stolperfalle #2: Ineffizientes LINQ

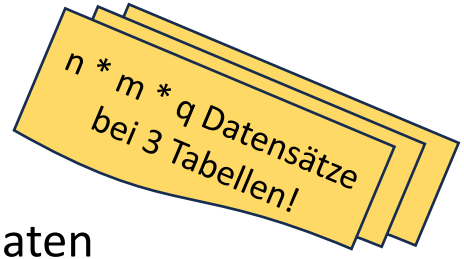
Problem:

- Verwendung von `ToList()` zu früh in der Query-Chain
- `.Contains()` auf In-Memory-Liste → Client-Side Evaluation
- Keine Übersetzung nach SQL möglich

Best Practice:

- Query-Ausdrücke sauber halten
- Logging aktivieren, um Client-Evaluation zu erkennen
- Übersetzbarkeit nach SQL prüfen
- IQueryable so lange wie möglich beibehalten

Stolperfalle #3: Eager Loading Overkill



Problem:

- `.Include()` mit mehreren Ebenen verwendet
  → Eine riesige Abfrage mit viel zu vielen Daten
- Kartesisches Produkt bei mehreren Collections

Best Practice:

- Selektives Laden: nur benötigte Daten
- **Split Queries** mit `AsSplitQuery()` verwenden
- DTOs für spezifische Use Cases
- Projection mit `.Select()` statt Include

Change Tracking & Performance

Stolperfalle #4: Unkontrolliertes Change Tracking

Problem:

- Hoher Speicherverbrauch bei Read-Only-Szenarien
- Change Tracker überwacht alle geladenen Entities
- Unerwartete Änderungen beim Speichern

Best Practice:

- `AsNoTracking()` für Read-Only-Queries
- Explizit `Update()` für Writes verwenden
- Bewusster Umgang mit Tracking-Verhalten

Stolperfalle #5: Falscher Umgang mit SaveChanges()

Problem:

- Loop mit 1000x `SaveChanges()` aufgerufen
- Jeder Aufruf = 1 Datenbank-Roundtrip
- Extrem langsame Performance

Best Practice:

- `SaveChanges()` gebündelt verwenden
- `SaveChangesAsync()` für asynchrone Verarbeitung
- Batch-Größen bei sehr großen Datenmengen beachten

Stolperfalle #6: Fehlende Bulk-Operationen

Problem:

- foreach-Loop mit 10.000 Updates/Deletes
- Jede Entity wird einzeln verarbeitet
- Ineffizient bei Massen-Operationen

Best Practice:

- `.ExecuteUpdate()` für Massen-Updates (EF Core 7+)
- `.ExecuteDelete()` für Massen-Deletes
- Ggf. `EFCore.BulkExtensions` NuGet-Package

DbContext Handling

Stolperfalle #7: DbContext-Lifecycle

Problem:

- Singleton-DbContext in Dependency Injection
- oder **EIN** DbContext für alles
- Memory Leak durch nicht freigegebene Entities
- Shared State zwischen Requests

Best Practice:

- Scoped-Lifetime in DI verwenden
- using-Pattern in einfachen Tools/Console-Apps
- DbContext nie als Singleton

Stolperfalle #8: Migrations-Chaos

Problem:

- Zwei Entwickler erzeugen parallel Migrations
- Konflikte beim Merge in Git
- Inkonsistente Migrations-Historie

Best Practice:

- Gemeinsame Konventionen im Team definieren
- `dotnet ef migrations remove` bei Konflikten
- “Baseline”-Migration bei bestehenden Projekten
- Migration-Script-Review im Pull Request

Modellierung

Stolperfalle #9: Value Objects & Owned Types

Problem:

- Owned Type führt zu unerwarteten Tabellenstrukturen
- Probleme beim Update von Owned Entities
- Komplexe JSON-Spalten nicht gewünscht

Best Practice:

- Owned Types bewusst designen
- Alternativen erwägen: Separate Entities mit 1:1-Beziehung
- Table-Splitting nur bei echten Value Objects
- Dokumentation der Entscheidung

Stolperfalle #10: Concurrency

Problem:

- Zwei User ändern denselben Datensatz gleichzeitig
- “Last Save Wins” überschreibt Änderungen
- ggf. Datenverlust ohne Fehlermeldung

Best Practice:

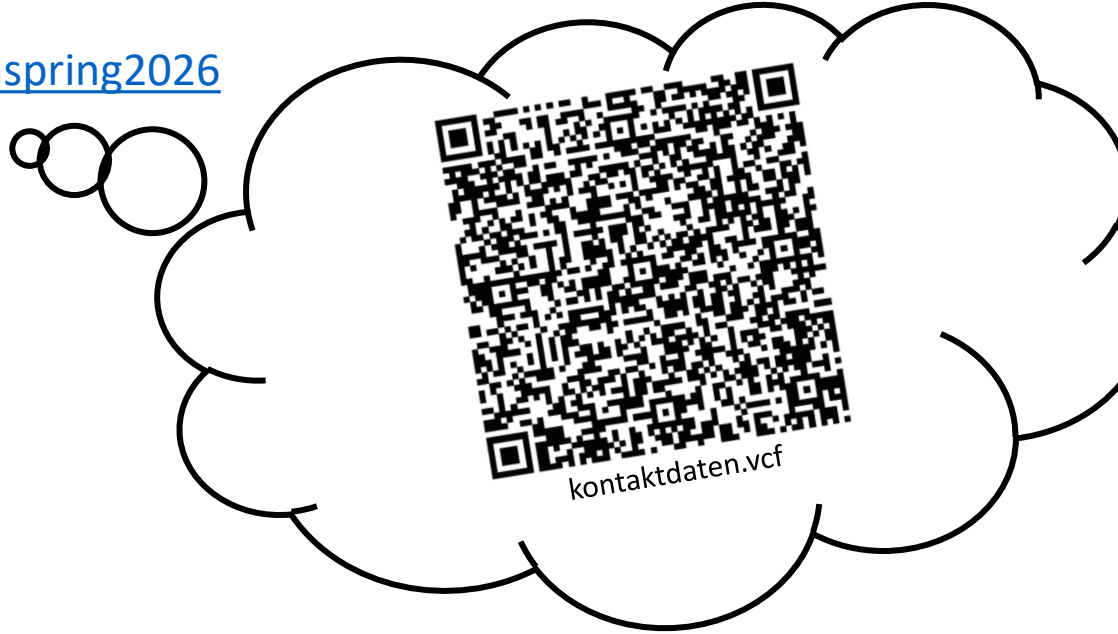
- Prüfen, ob problematisch
- `RowVersion`/`ConcurrencyToken` verwenden
- Exception-Handling für `DbUpdateConcurrencyException`
- Retry-Logik oder Konflikt-Auflösung implementieren

Bitte

nicht vergessen

zu bewerten!

entwickler.de

Vielen Dank!

Slides und Code-Sample auf
https://github.com/olaflischke/bastaspring2026